

Homework 2: Ghostbusters and Bayes Nets

1. Bayes Net Structure

Objective: Implement `constructBayesNet` function in `inference.py` to create an empty Bayes Net structure as described.

Figure 1 below shows the implementation of the network by adding the variables and edges as required i.e., “Ghost 0 $\rightarrow$ Obs 0 $\leftarrow$ Pacman $\rightarrow$ Obs 1 $\leftarrow$ Ghost 1”.

```
PAC = "Pacman"
GHOST0 = "Ghost0"
GHOST1 = "Ghost1"
OBS0 = "Observation0"
OBS1 = "Observation1"
X_RANGE = gameState.getWalls().width
Y_RANGE = gameState.getWalls().height
MAX_NOISE = 7

variables = [PAC, GHOST0, GHOST1, OBS0, OBS1]
edges = [(PAC, GHOST0), (PAC, GHOST1), (GHOST0, OBS0), (GHOST1, OBS1)]
variableDomainsDict = {}

# Set the variable domains for each variable
variableDomainsDict[PAC] = [(x, y) for x in range(X_RANGE) for y in range(Y_RANGE)]
variableDomainsDict[GHOST0] = [(x, y) for x in range(X_RANGE) for y in range(Y_RANGE)]
variableDomainsDict[GHOST1] = [(x, y) for x in range(X_RANGE) for y in range(Y_RANGE)]
# The distance is the maximum Manhattan distance from the origin to the farthest point
distance = max(manhattanDistance((0, 0), (x, y)) for x in range(X_RANGE) for y in range(Y_RANGE))
# The observation domain is the range of possible noisy distances
obs_domain = list(range(distance + MAX_NOISE + 1))
variableDomainsDict[OBS0] = obs_domain
variableDomainsDict[OBS1] = obs_domain

net = bn.constructEmptyBayesNet(variables, edges, variableDomainsDict)
return net
```

Figure 1: Implementation for 'constructBayesNet' in `inference.py`

2. Join Factors

Objective: Implement `joinFactors` that takes a list of factors and returns their product; Operate even on unnormalized tables

Figure 2 below shows the implementation 'joinFactors' method, figure 3 shows the output of the implemented function running a test where Factor A, $P(D|W)$ and Factor B, $P(W)$ are passed to implemented 'joinFactors($P(D|W)$, $P(W)$) to compute $P(D, W)$.

```

factors = list(factors)

# typecheck portion
setsOfUnconditioned = [set(factor.unconditionedVariables()) for factor in factors]
if len(factors) > 1:
    intersect = functools.reduce(lambda x, y: x & y, setsOfUnconditioned)
    if len(intersect) > 0:
        print("Factor failed joinFactors typecheck: ", factor)
        raise ValueError("unconditionedVariables can only appear in one factor. \n"
            + "unconditionedVariables: " + str(intersect) +
            "\nappear in more than one input factor.\n" +
            "Input factors: \n" +
            "\n".join(map(str, factors)))

variableDomainsDict = factors[0].variableDomainsDict()
# Get the set of unconditioned and conditioned variables
unconditionedVariables = set()
conditionedVariables = set()
# Iterate through the factors to get the unconditioned and conditioned variables
for f in factors:
    unconditionedVariables |= set(f.unconditionedVariables())
    conditionedVariables |= set(f.conditionedVariables())
# Remove the join variable from the unconditioned variables
conditionedVariables -= unconditionedVariables
newFactor = Factor(list(unconditionedVariables), list(conditionedVariables), variableDomainsDict)
# Iterate through all possible assignments for the new factor
for assignment in newFactor.getAllPossibleAssignmentDicts():
    prob = 1.0 # Initialize probability to 1
    for f in factors:
        prob *= f.getProbability(assignment)
    newFactor.setProbability(assignment, prob)

return newFactor

```

Figure 2: Implementation for 'joinFactors' method

<pre> tracking copy > test_cases > q2 > 1-product-rule.test 1 # test_cases/q3/1-product-rule.test. 2 variableDomainsDict: "" 3 W : sun rain 4 D : wet dry 5 "" 6 7 # endOfNonFactors 8 9 AunconditionedVariables: "D" 10 11 AconditionedVariables: "W" 12 13 AFactorTable: "" 14 D : wet, W : sun = 0.1 15 D : dry, W : sun = 0.9 16 D : wet, W : rain = 0.7 17 D : dry, W : rain = 0.3 18 "" 19 20 21 BunconditionedVariables: "W" 22 23 BconditionedVariables: "" 24 25 BFactorTable: "" 26 W : sun = 0.8 27 W : rain = 0.2 28 "" 29 30 </pre>	<pre> tracking copy > test_cases > q2 > 1-product-rule.solution 1 # This is the solution file for test_cases/q3/1-product-rule.test. 2 variableDomainsDict: "" 3 D : wet dry 4 W : sun rain 5 "" 6 7 unconditionedVariables: "D W" 8 9 conditionedVariables: "" 10 11 FactorTable: "" 12 D : wet, W : sun = 0.08 13 D : dry, W : sun = 0.72 14 D : wet, W : rain = 0.14 15 D : dry, W : rain = 0.06 16 "" 17 18 19 </pre>
---	--

Figure 3: Output for 'joinFactors(P(D|W), P(W))'

3. Eliminate Variable

Objective: Implement eliminate to sum out a variable from a factor

Figure 4 below shows the implementation 'eliminate' method, figure 5 below shows the output of the implemented function running a test where Factor A, P(D,W) and variable W is eliminated by passing to implemented 'eliminate(P(D,W), W) to compute P(D).

```
# typecheck portion
if eliminationVariable not in factor.unconditionedVariables():
    print("Factor failed eliminate typecheck: ", factor)
    raise ValueError("Elimination variable is not an unconditioned variable " \
        + "in this factor\n" +
        "eliminationVariable: " + str(eliminationVariable) + \
        "\nunconditionedVariables: " + str(factor.unconditionedVariables()))

if len(factor.unconditionedVariables()) == 1:
    print("Factor failed eliminate typecheck: ", factor)
    raise ValueError("Factor has only one unconditioned variable, so you " \
        + "can't eliminate \nthat variable.\n" + \
        "eliminationVariable: " + str(eliminationVariable) + "\n" + \
        "unconditionedVariables: " + str(factor.unconditionedVariables()))

# Calculate the set of unconditioned and conditioned variables
newUnconditionedVariables = set(factor.unconditionedVariables())
newUnconditionedVariables.remove(eliminationVariable)
newConditionedVariables = set(factor.conditionedVariables())

newFactor = Factor(list(newUnconditionedVariables), list(newConditionedVariables), factor.variableDomainsDict())
# Iterate through all possible assignments for the new factor
for assignment in newFactor.getAllPossibleAssignmentDicts():
    prob = 0.0 # Initialize probability to 0
    # Iterate through all possible assignments for the elimination variable
    for eliminationAssignment in factor.variableDomainsDict()[eliminationVariable]:
        newAssignment = assignment.copy()
        newAssignment[eliminationVariable] = eliminationAssignment
        prob += factor.getProbability(newAssignment) # Sum the probabilities
    newFactor.setProbability(assignment, prob)

return newFactor

urn eliminate
```

Figure 4: Implementation for 'eliminate' method

<pre>tracking copy > test_cases > q3 > 1-simple-eliminate.test 1 # test_cases/q4/1-simple-eliminate.test. 2 variableDomainsDict: "" 3 W : sun rain 4 D : wet dry 5 "" 6 7 # endOfNonFactors 8 9 AunconditionedVariables: "D W" 10 11 AconditionedVariables: "" 12 13 AFactorTable: "" 14 D : wet, W : sun = 0.08 15 D : dry, W : sun = 0.72 16 D : wet, W : rain = 0.14 17 D : dry, W : rain = 0.06 18 "" 19 20</pre>	<pre>tracking copy > test_cases > q3 > 1-simple-eliminate.solution 1 # test_cases/q4/1-simple-eliminate.test. 2 variableDomainsDict: "" 3 D : wet dry 4 W : sun rain 5 "" 6 7 unconditionedVariables: "D" 8 9 conditionedVariables: "" 10 11 FactorTable: "" 12 D : wet = 0.22 13 D : dry = 0.78 14 "" 15 16 17</pre>
--	--

Figure 5: "eliminate(P(D, W), W)" to compute P(D)

4. Variable Elimination

Objective: Perform probabilistic inference via variable elimination

Figure 6 below shows the implementation for the inference by variable elimination. For each hidden variable in elimination order, all factors are joined that mention that variable, if the joined factor has more than one unconditional variable. Finally normalize is called so that the table sums to 1. Figure 7 shows the output for the test the problem where factor 1 $P(D|W)$, factor 2 $P(W)$ and factor 3 $P(T|W)$ to compute $P(D|W = \text{sun})$.

```
# Initialize the list of factors with evidence
factors = bayesNet.getAllCPTsWithEvidence(evidenceDict)

#Eliminate each variable in order
for var in eliminationOrder:
    factors, joinedFactor = joinFactorsByVariable(factors, var)
    if len(joinedFactor.unconditionedVariables()) > 1:
        # If the joined factor has more than one unconditioned variable, eliminate it
        factors.append(eliminate(joinedFactor, var))
    else:
        pass # If it has only one unconditioned variable, just discard it

if len(factors) > 0:
    return normalize(joinFactors(factors))
else:
    return None

return inferenceByVariableElimination
```

Figure 6: Implementation for inference by variable elimination

```

1 variables: ""
2 W
3 D
4 T
5 ""
6
7 edges: ""
8 W D
9 W T
10 ""
11
12 variableDomainsDict: ""
13 W : sun rain
14 D : dry wet
15 T : hot cold
16 ""
17
18 queryVariables: "D"
19
20 evidenceDict: ""
21 W : sun
22 ""
23
24 variableEliminationOrder: "T"
25
26 # endOfNonFactors
27
28 DunconditionedVariables: "D"
29
30 DconditionedVariables: "W"
31
32 DFactorTable: ""
33 D : dry, W : sun = 0.9
34 D : wet, W : sun = 0.1
35 D : dry, W : rain = 0.3
36 D : wet, W : rain = 0.7
37 ""
38
39
40 WunconditionedVariables: "W"
41
42 WconditionedVariables: ""
43
44 WFactorTable: ""
45 W : sun = 0.8
46 W : rain = 0.2
47 ""
48
49
50 TunconditionedVariables: "T"
51
52 TconditionedVariables: "W"
53
54 TFactorTable: ""
55 T : hot, W : sun = 0.75
56 T : cold, W : sun = 0.25
57 T : hot, W : rain = 0.1
58 T : cold, W : rain = 0.9
59 ""

```

```

1 # This is the solution file for test_cases/q6/1-disconnected-eliminate.test.
2 variableDomainsDict: ""
3 D : dry wet
4 W : sun
5 T : hot cold
6 ""
7
8 unconditionedVariables: "D"
9
10 conditionedVariables: "W"
11
12 FactorTable: ""
13 D : dry, W : sun = 0.9
14 D : wet, W : sun = 0.1
15 ""
16
17
18 callTrackingList: "[('join', 'T')]"
19

```

Figure 7: Compute $P(D|W = \text{sun})$

5a, 5b: Distributions and Noisy Observations

5a: Implement `DiscreteDistribution` to extend the Python dictionary, where keys are elements of the distribution, and values are the associated weights.

5b: Calculates the probability of a noisy distance reading between Pacman and a ghost. Noisy distance reading probabilities

Figure 8 below shows the implementation of ‘`DiscreteDistribution`’ where keys are elements of the distribution and values are the associated weights. Figure 9 below shows the implementation to return the probability of ‘ $P(\text{noisy}|\text{trueDist})$ ’.

```

    """
    YOUR CODE HERE
    """
    # Get the total sum of the distribution
    total = float(sum(self.values()))
    if total == 0:
        return # do nothing if total is 0
    # Normalize each key by dividing its value by the total
    for key in list(self.keys()):
        self[key] /= total

```

Figure 8: Implementation for 'DiscreteDistribution' method

```

"""
Return the probability P(noisyDistance | pacmanPosition, ghostPosition).
"""
""" YOUR CODE HERE """
# Get the jail position
if ghostPosition == jailPosition:
    return 1.0 if noisyDistance is None else 0.0 # Ghost is in jail
# Calculate the Manhattan distance between Pacman and the ghost

if noisyDistance is None:
    return 0.0 # No observation

return busters.getObservationProbability([noisyDistance, manhattanDistance(pacmanPosition, ghostPosition)])
"""
"""

```

Figure 9: Calculate 'P(noisy|trueDist)'

6. Exact Inference - Observation

Task: Implement observeUpdate to update ghost position beliefs based on Pacman's observations ; Bright colors represent higher beliefs

Figure 10 below shows the implementation of the observeUpdate method. Figure 11 demonstrates how beliefs start broad and narrow down as more evidence is collected. Initially the left side of figure 11 shows multiple beliefs with bright colors representing higher beliefs; as more evidence is collected (right side of figure 11) shows narrowed observations.

```

# Get the jail position and the current pacman position
pacmanPosition = gameState.getPacmanPosition()
jailPosition = self.getJailPosition()

updatedBeliefs = DiscreteDistribution()
for ghostPosition in self.allPositions:
    # P( noisyDistance | PacmanPosition, GhostPosition )
    prob = self.getObservationProb(observation, pacmanPosition, ghostPosition, jailPosition)
    updatedBeliefs[ghostPosition] = prob * self.beliefs[ghostPosition]

self.beliefs = updatedBeliefs
self.beliefs.normalize()

```

Figure 10: Implementation for 'observeUpdate' method

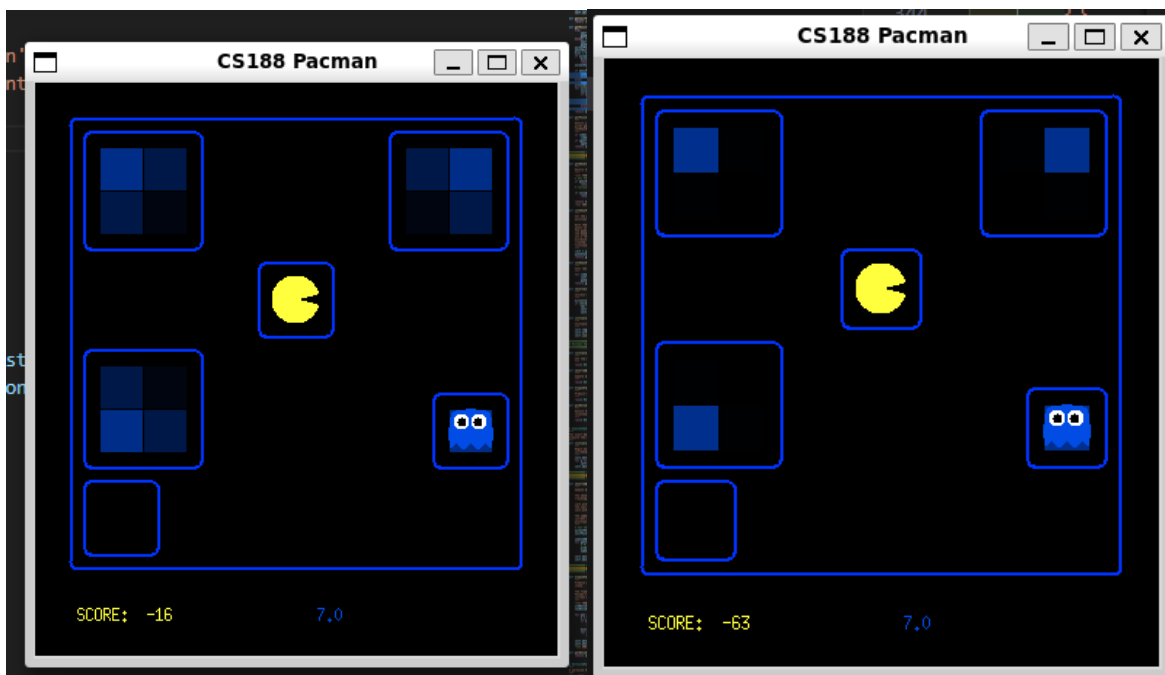


Figure 11: Broad beliefs initially (left), beliefs narrow down as more evidence is collected (Right)

Question 7: Exact Inference - Time Elapse

Task: Implement `elapseTime` to update beliefs based on possible ghost motion (unobserved); Ghost motion should reflect tendencies like `GoSouthGhost` behavior

Figure 12 below shows the implementation of `elapseTime` to update the beliefs based on the ghost motion.

```

oldBeliefs = self.beliefs
newBeliefs = DiscreteDistribution()

for pos in self.allPositions: # Iterate through all possible positions
    w = oldBeliefs[pos]
    if w == 0: # Skip if the weight is zero
        continue

    dist = self.getPositionDistribution(gameState, pos) # Get the distribution of the next position
    for newPos, prob in dist.items(): # Iterate through the next positions
        newBeliefs[newPos] = newBeliefs.get(newPos, 0) + w * prob # Update the new beliefs

self.beliefs = newBeliefs
self.beliefs.normalize()

```

Figure 12: Implementation for 'elapsedTime' method

Question 8: Full Inference Test

Objective: Use belief distributions to: Identify most likely ghost positions; Select actions that minimize maze distance to nearest ghost

Figure 13 below shows the implementation for full inference test which identifies the most likely ghost positions and takes actions accordingly to the nearest ghost. Figure 14 shows the visualization provided using these methods where the agent now moves towards the ghost nearest to it and then eats it. Finally, figure 15 shows all the ghosts eaten by the Pacman agent.

```

# Find the most likely position for each living ghost
mostLikelyGhostPositions = [ dist.argmax() for dist in livingGhostPositionDistributions ]
# Find the closest ghost to Pacman
closestGhostPosition = min(mostLikelyGhostPositions, key=lambda pos: self.distancer.getDistance(pacmanPosition, pos))
# Find the best action that brings Pacman closest to the closest ghost
bestAction = min(legal, key=lambda action: self.distancer.getDistance(Actions.getSuccessor(pacmanPosition, action), closestGhostPosition))
return bestAction

```

Figure 13: Implementation for full inference test

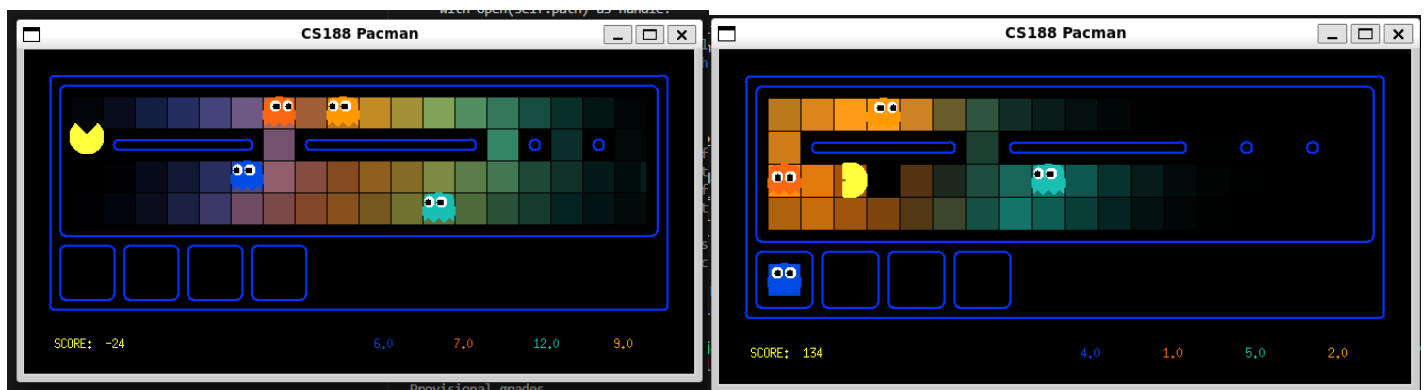


Figure 14: Pacman agent moving towards closest ghost and current belief distributions

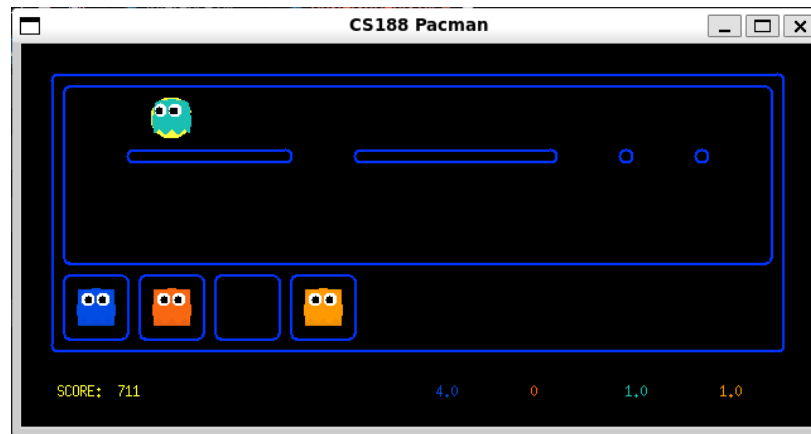


Figure 15: Pacman agent eats all ghosts

Question 9: Particle Filter - Initialization

Task: Implement initialize Uniformly to spread particles across legal positions; Implement getBeliefDistribution to create a distribution from current particles

Figure 16 below shows the implementation to uniformly spread particles across legal positions and then create a distribution.

```
def initializeUniformly(self, gameState: busters.GameState):
    """
    Initialize a list of particles. Use self.numParticles for the number of
    particles. Use self.legalPositions for the legal board positions where
    a particle could be located. Particles should be evenly (not randomly)
    distributed across positions in order to ensure a uniform prior. Use
    self.particles for the list of particles.
    """
    self.particles = [] # Initialize the list of particles
    for i in range(self.numParticles): # For each particle
        self.particles.append(self.legalPositions[i % len(self.legalPositions)]) # Assign a legal position to the particle

def getBeliefDistribution(self):
    """
    Return the agent's current belief state, a distribution over ghost
    locations conditioned on all evidence and time passage. This method
    essentially converts a list of particles into a belief distribution.

    This function should return a normalized distribution.
    """
    distribution = DiscreteDistribution() # Initialize the distribution
    for p in self.particles: # For each particle
        distribution[p] = distribution.get(p, 0) + 1 # Increment the count for the particle's position
    distribution.normalize() # Normalize the distribution
    return distribution
```

Figure 16: Implementation for 'initializeUniformly' and 'getBeliefDistribution'

Question 10: Particle Filter - Observation Update

Task: Implement observeUpdate to update particles using noisy sensor readings

Figure 17 below shows the implementation for observeUpdate method to update particles using sensor readings.

```
# Get the jail position and the current pacman position
pacmanPosition = gameState.getPacmanPosition()
jailPosition = self.getJailPosition()
weights = DiscreteDistribution() # Initialize the weights

for p in self.particles: # For each particle
    prob = self.getObservationProb(observation, pacmanPosition, p, jailPosition) # Get the probability of the observation given the particle's position
    weights[p] = weights.get(p, 0) + prob # Update the weight for the particle's position

if weights.total() == 0: # If all weights are zero
    self.initializeUniformly(gameState) # Reinitialize the particles
    return

# Normalize the weights
weights.normalize()
newParticles = [ weights.sample() for _ in range(self.numParticles) ]
self.particles = newParticles
```

Figure 17: Implementation for 'observeUpdate' method

Question 11: Particle Filter - Time Elapse

Task: Implement elapseTime to simulate particle transitions between steps

Figure 18 shows the implementation of elapseTime to simulate particle transitions between time steps. Figures 19 and 20 demonstrate the visualization of the same methods implemented and represented over a time. Figure 19 initially shows the beliefs for ghost during initial time periods and then moves to the nearest ghost and then figure 20 shows the updated beliefs after the first ghost has been eaten and the agent starts moving towards the second ghost.

```
def elapseTime(self, gameState):
    """
    Sample each particle's next state based on its current state and the
    gameState.
    """
    newParticles = [] # Initialize the new particles

    for oldParticle in self.particles: # For each old particle
        dist = self.getPositionDistribution(gameState, oldParticle) # Get the distribution of the next position
        newParticles.append(dist.sample()) # Sample a new position from the distribution
    self.particles = newParticles
```

Figure 17: Implementation for elapseTime method

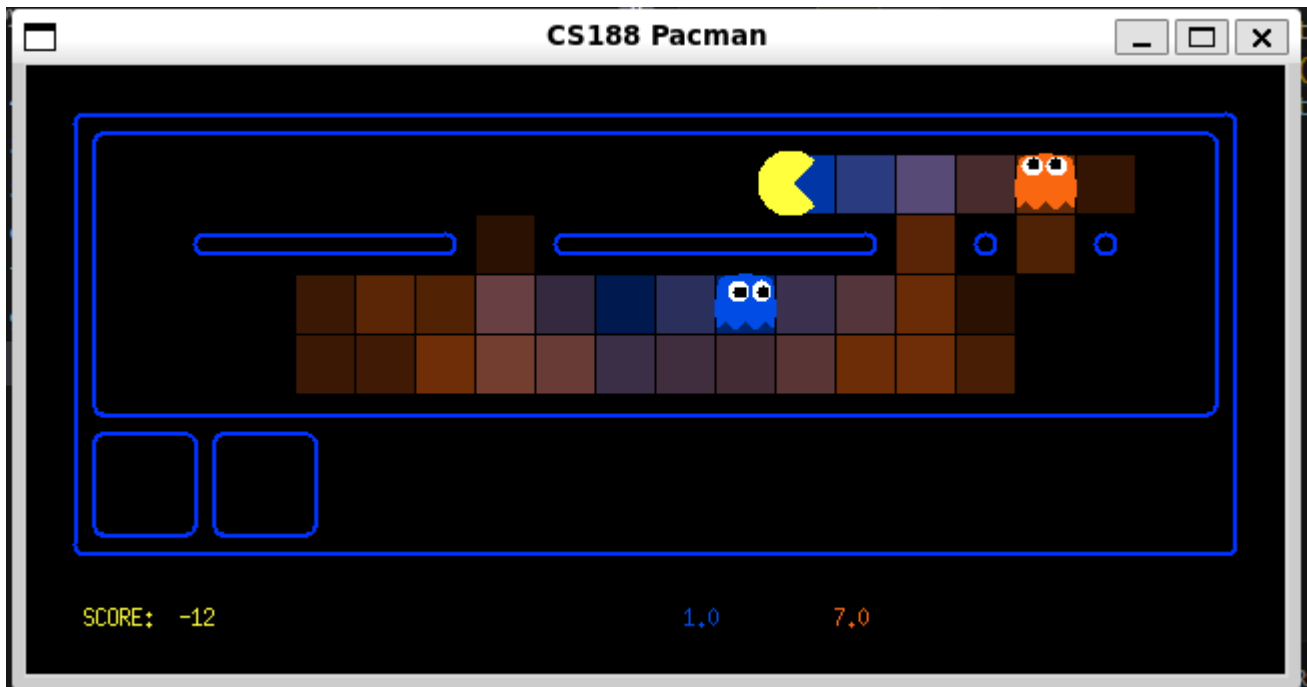


Figure 19: beliefs for ghost during initial time periods, and moving towards nearest ghost

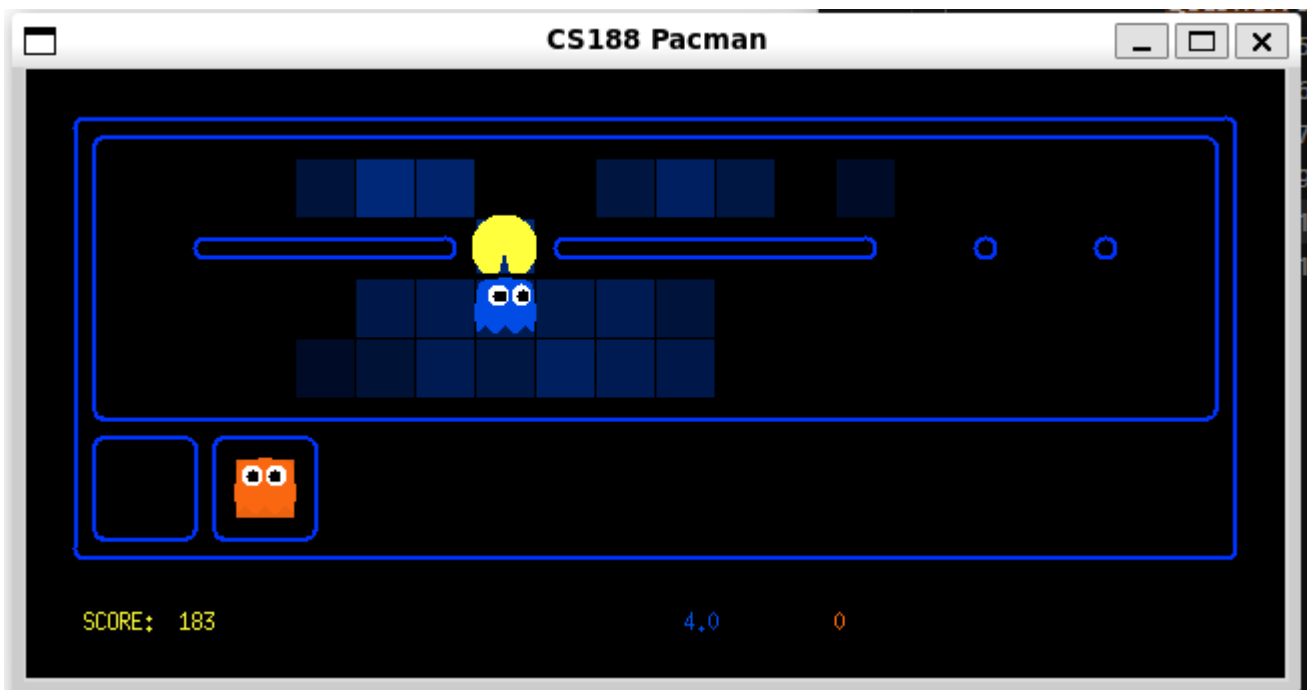


Figure 20: beliefs for ghost after some time, post eating the first ghost

Note: All the code is included in the attached zip file, 'codeArchive.zip'. This includes a shell script added which has python commands required to generate the visualizations shown above. (shown as in below figure)



References

<https://inst.eecs.berkeley.edu/~cs188/sp24/projects/proj4/>